SYSTEMS OVERVIEW

Triage Engine

An event-processing service built around a single requirement: that no piece of work is ever lost quietly. What it does, how it holds that promise when things fail, and where else the pattern applies.

Younes Kaouani

triage-engine.youneskaouani.dev

github.com/younesKAOUANI/triage-engine

01 — WHAT IT IS

A service that accepts work and refuses to lose it

In its current form it triages support tickets: it accepts them, classifies each one with a language model, and notifies a downstream system. That description undersells it, because the classification is roughly four hundred lines and the rest of the system exists to survive everything that can go wrong around it.

The service exposes one meaningful entry point. A client posts an event — a subject, a body, an optional email address. The response comes back in milliseconds with an identifier and a `202`, because the work has been *accepted*, not completed. Moments later the item has a category, a priority, a one-line summary and a confidence score, and a downstream system has been told about it exactly once.

Between those two moments sits the part that matters. The event is written to a database alongside a record of the key it arrived under. A job is queued. A worker picks it up, calls a language model, and writes the result and the outbound notification in a single transaction. A separate process delivers that notification, retrying until the other side acknowledges it.

Every one of those steps can fail, and several of them can fail in ways that are invisible unless you plan for them. The caller can send the same event twelve times. Two copies can arrive in the same millisecond. The worker can die halfway through. The model can time out, or return confident nonsense that does not match the expected shape. The downstream system can return errors for an hour.

None of those situations may result in a duplicate, and none of them may result in something disappearing without a trace. That constraint is the product. Everything in the architecture follows from it.

> The organising principle is **nothing is ever silently lost**. Both halves carry weight. Work must always end up somewhere durable and queryable. And when giving up is genuinely unavoidable, it must be visible — every abandonment path increments a counter built to be alerted on.

02 — THE PROBLEM

Why this is harder than it looks

Systems that send you events retry when they do not hear back quickly enough. They cannot tell the difference between a request that was lost and a response that was lost, so they resend. This is not an edge case; it is the normal operating mode of every webhook, message broker and integration partner you will ever work with.

At-least-once delivery is the default, everywhere

The consequence is that your handler will be called more than once for the same logical event, sometimes concurrently. If handling it has any side effect — a charge, an email, a row, a downstream call — then handling it twice does that side effect twice. The naive fix is to check whether you have seen the event before, and the naive check has a race in it: two requests both look, both find nothing, and both proceed.

Then the middle of your pipeline fails

Meanwhile the interesting work is slow and unreliable. A model call takes seconds and sometimes never returns. Doing it inside the request would tie up a connection and time out the caller, so it moves to a queue — and now there is a gap between "we accepted this" and "we finished it" in which the process can restart. Whatever was in flight has to be recoverable from durable state, not from memory.

And the failures are the quiet kind

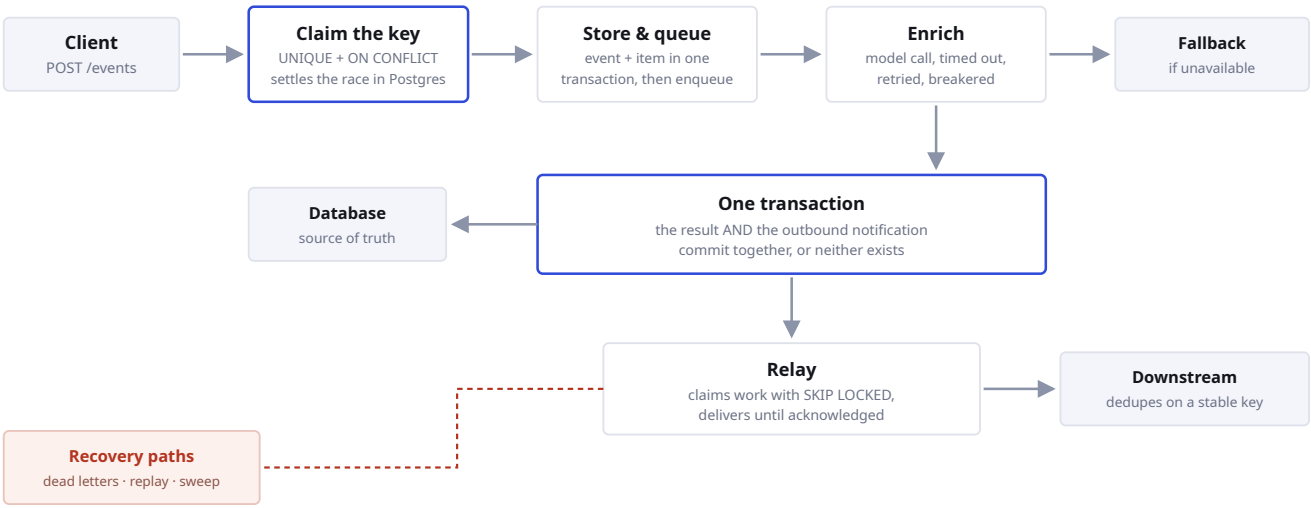
The failure that matters is not the one that returns a 500. It is the one where the database says the item was processed, the downstream system never heard about it, and nothing anywhere records the discrepancy. Nobody notices until a customer asks why they never got a reply. By then the evidence is gone.

WHAT GOES WRONG	WHAT IT COSTS IF UNHANDLED
Duplicate delivery	The same work performed twice, with every side effect repeated.
Two copies at once	A race that a naive "have I seen this?" check does not close.
Crash mid-processing	Half-written state, and an item that no longer appears anywhere as outstanding.
Commit succeeds, notify fails	Internally complete, externally invisible. The hardest class to detect.
Dependency unavailable	Either the item is dropped, or the whole intake stalls behind it.
Retries exhausted	The item vanishes from the queue with no durable record of why.

03 — HOW IT WORKS

The path an event takes

Five stages, each with a specific job. The interesting design is in the boundaries between them rather than in any one stage.



What each boundary is doing

STAGE	THE DECISION IT MAKES
Claim the key	Whether this is new work, a duplicate of finished work, a duplicate of work in progress, or the same key being reused for different content. Decided by the database, not by application code, so it stays correct under concurrency and across any number of instances.
Store & queue	The raw event and the derived item are written together. The job identifier is the caller's own key, so re-delivery cannot enqueue the same work twice.
Enrich	Call the model with a hard timeout, retry with jitter, and stop calling it entirely once a circuit breaker sees it failing. Validate the response against a schema — valid JSON is not the same as correct content.
One transaction	The result and the outbound notification commit together. This is the step that removes the "saved but never announced" failure entirely, rather than making it less likely.
Relay	Claims pending notifications with a lock that lets multiple instances work the same table without coordinating, and delivers with retries against a stable key the receiver can deduplicate on.

04 — GUARANTEES

What it promises, and what enforces it

Each promise below is held by a specific mechanism, not by convention or by care. That distinction matters: the point of building it this way is that the guarantee survives a developer who has never read this document.

PROMISE	WHAT ENFORCES IT
The same event never produces two items	A unique constraint plus a conditional insert. Exactly one of any number of simultaneous callers wins; the rest are told the work is already done or already running.
Reusing a key for different content is refused	The key is matched against a hash of the content, so a genuine client bug surfaces as a conflict rather than being silently absorbed.
A completed item is always announced	The notification row is written in the same transaction as the result. There is deliberately no way to emit one outside a transaction, so the pattern cannot be bypassed by accident.
Notifications survive a crash	Delivery happens inside the transaction that marks the row sent. A process that dies mid-flight rolls back and redelivers later.
Work that fails permanently is kept	It lands in a queryable table with the original payload, the error, the stack and the attempt count — and can be replayed through its original key, so replaying cannot duplicate anything.
An unavailable dependency does not stop intake	A deterministic fallback produces a usable result immediately, flagged with low confidence, and a better attempt is queued for when the dependency recovers.
Nothing stays stuck	A periodic sweep treats the database as the source of truth and re-drives anything left in a degraded state, closing the case where the recovery job itself was lost.

> Giving up is sometimes correct. A downstream system that has rejected a notification twenty times may genuinely be gone. What is never acceptable is giving up **without saying so** — so each of those paths increments a counter that exists specifically to be alerted on.

05 — UNDER FAILURE

What actually happens when things break

The behaviour below is not aspirational. Each case is covered by an automated test that runs the whole system against a real database and a real queue, with only the outbound network calls faked.

The model is down

Calls are timed out and retried a few times with jitter. If failures continue, a circuit breaker opens and further calls are skipped entirely rather than piling up against something known to be broken. The item is classified by a deterministic rule-based classifier instead, marked as a degraded result with low confidence, and a second attempt is scheduled for later on a staggered delay so that recovery does not arrive as a stampede. **Intake never stops.** The model affects quality, never availability.

The model returns something wrong-shaped

Every response is validated against a strict schema. A response that is valid JSON but has an unexpected category, a missing field or an out-of-range confidence is treated exactly like a network failure: retried, then degraded. Bad data never reaches storage.

A worker dies mid-item

The queue returns the job to the pool. Because the item's identity comes from the caller's key rather than from anything in memory, the retry reprocesses the same item rather than creating a second one. If it exhausts its attempts — including the several ways a queue can retire a job early, which are easy to miss and were the subject of a specific fix — it is recorded as a dead letter with full context rather than disappearing.

The downstream system is failing

Notifications accumulate as pending rows and are retried on a backoff. A growing pending count is visible as a gauge. If one exhausts its attempts it is marked terminal and a counter increments — the deliberate, visible form of giving up.

Someone abuses the public endpoint

Writes are rate limited per caller. Health checks, metrics and the internal delivery target are exempt, because a throttled health check reads as an outage and a throttled delivery target would throttle the system's own notifications. Data past a retention window is pruned automatically, and only settled records are eligible — anything still in flight is never removed regardless of age.

06 — APPLICATIONS

Where else this shape fits

Support tickets are an illustration, not the point. Strip the domain away and what remains is a general pattern, and the pattern is worth recognising because a surprising number of production systems are instances of it without anyone having named it.

> **Accept an event you did not originate and cannot trust to arrive once. Do something slow and failure-prone with it. Then tell another system, exactly once, that it happened.** Anywhere those three sentences are true, the machinery here applies almost unchanged.

What changes between domains is small and well isolated: the processing step sits behind a single interface, and the notification target is a URL. Everything else — the deduplication ledger, the transactional handoff, the dead-letter table, the recovery sweep — is domain-agnostic.

WHERE THE MACHINERY EARNS ITS COMPLEXITY

STRONG FIT

Payment and billing webhooks

Payment providers retry aggressively and make no promise about delivering once. Every provider's own documentation tells you to deduplicate on the event id, which is precisely the ledger here.

Processing a charge twice refunds twice, fulfils twice, or posts the same amount to a ledger twice. Unwinding it is manual and customer-visible.

Swap the processing step for reconciliation or ledger posting; point the notification at the order system.

STRONG FIT

Order and fulfilment intake

Orders arrive from storefronts, marketplaces and partner APIs that resend on timeout. The processing step — pricing, stock allocation, fraud scoring — is slow and depends on services that fail independently.

A duplicate order ships twice and decrements stock twice. A lost notification means a paid order the warehouse never sees.

Enrichment becomes allocation and scoring; the notification goes to the warehouse or ERP.

STRONG FIT

Document intake: claims, KYC, invoices

Extraction is slow, occasionally wrong, and increasingly done by a model. The immutable record of the original submission and the ability to replay a failure are regulatory requirements here, not conveniences.

A submission that disappears between upload and review is a compliance incident, and one nobody can reconstruct after the fact.

Enrichment becomes OCR plus extraction; the low-confidence path routes to human review instead of a rule-based guess.

STRONG FIT

Moderation and classification queues

The clearest match for the degrade-then-upgrade behaviour. When the model is unavailable you do not want intake to stop — you want a conservative provisional decision now and the real one shortly after.

Blocking on the model means submissions queue unreviewed; skipping it means unreviewed content goes live.

The fallback becomes "hold pending review" rather than a keyword classifier.

GOOD FIT

Third-party webhook landing zone

A single hardened front door for inbound integrations — accept quickly, acknowledge, then do the real work behind a queue with retries you control rather than the sender's.

Doing the work inline means the sender times out and resends, multiplying the load exactly when the system is already slow.

Usually nothing structural. Add signature verification at ingestion and fan out by event type.

GOOD FIT

Outbound notification and fan-out

The delivery half is a general solution to "a thing happened in my database and another system must hear about it exactly once", independent of the rest of the pipeline.

Sending from application code after a commit loses the message whenever the process dies in the gap between the two.

Adopt the outbox and relay alone; the ingestion side is not required for it to be useful.

Where it would be the wrong choice

Two cases are worth naming, because the pattern is easy to over-apply. **High-volume telemetry** — clickstreams, device readings, log shipping — writes a durable row and runs a transaction per event, which is far too heavy above a few hundred events per second; a stream processor is the right tool and duplicate readings usually do not matter anyway. And **anything requiring strict ordering per entity** is not served by this at all: items are processed concurrently, so "apply these three updates to account X in sequence" needs partitioning the engine does not have.

What you would keep, and what you would replace

COMPONENT	DOMAIN-SPECIFIC?
Deduplication ledger	No. Identical in every application above.
Transactional handoff	No. The mechanism is the same whatever the payload.
Delivery relay	No. Only the destination URL changes.
Dead letters and replay	No. The stored payload is opaque to it.
Recovery sweep	Mostly. It scans for a specific stalled state, which you would name for your own workflow.
The processing step	**Yes.** This is the part you write. It sits behind one interface, and substituting it is a binding change rather than a rewrite.
The fallback	**Yes.** What "degraded but usable" means is a domain judgement — a cautious guess, a hold, or a human queue.

07 — LIMITS

What it is not, and when not to reach for it

A description of a system that only lists strengths is marketing. These are the boundaries a competent reviewer would find within an hour, stated plainly.

Deliberate scope decisions

- **It is not a workflow engine.** The pipeline is one linear path. Branching, human approval steps, compensation and long-running sagas are out of scope; something like Temporal exists for that.
- **Ordering is not guaranteed.** Items are processed concurrently. Where strict per-entity sequencing matters, this needs partitioning it does not currently have.
- **Delivery is at-least-once, not exactly-once.** The receiver must deduplicate on the supplied key. That is a genuine requirement placed on the consumer, not a detail to skip past.
- **There is no tenancy model.** One noisy source can consume capacity that others need. Per-tenant partitioning and backpressure would both be required before serving multiple customers from one deployment.
- **Reads are not access-controlled.** Anyone holding an item's identifier can retrieve its contents. Acceptable for a demonstration; unacceptable for real personal data without authentication in front.

Known trade-offs

- The relay holds a row lock across the outbound call. Correct and simple at moderate volume, and the wrong shape at high volume, where the fix is to claim with a lease and deliver outside the transaction.
- The recovery sweep runs on every instance, so replicas duplicate the scan. Harmless but wasteful; a leader election would settle it.
- Processing is one item per call. Batching would reduce both cost and latency where the underlying dependency supports it.

When to reach for something simpler

If the work is fast, idempotent by nature, and has no side effect outside your own database, this is more machinery than the problem deserves. A plain handler and a unique constraint will do. The complexity here earns its place only when the processing step is genuinely unreliable *and* the side effect is genuinely hard to undo. When only one of those is true, take half the pattern.

08 — SEEING IT WORK

A running deployment, not a description

The system is deployed and public. The page below is not a marketing site; it drives the real service and shows what comes back.

WHERE	WHAT YOU GET
Interactive demo	**triage-engine.youneskaouani.dev** — submit an item and watch it move through the pipeline, then use the buttons that deliberately try to break the guarantees: resend the identical request, reuse a key with different content, fire twelve simultaneous copies, flood the endpoint. Each reports what came back and why it held.
Live signals	`/metrics` exposes the counters and gauges described throughout this document, including the two that indicate work was abandoned.
Failure records	`/dlq` lists anything that failed permanently, with the context needed to decide what to do about it.
Source and reasoning	The repository contains the full architecture reference and nine decision records covering the choices that were genuinely contested, including the ones that turned out to be wrong.

How it is built and run

TypeScript on NestJS, PostgreSQL for state, Redis for the queue, a language model for classification. It runs as containers behind a reverse proxy on a single host; the datastores have no route off the machine. Every push to the main branch runs the full test suite against a real database and queue, builds an image, deploys it pinned to that exact commit, and then verifies the new version is serving before the run is allowed to report success.

Written by Younes Kaouani. The claims in this document about behaviour under failure correspond to automated tests in the repository; where something is aspirational or unbuilt, it is described as such in section 07.